



uOttawa

CEG 4195

**Final Project**

Student Number

Name

300314847

Ian Worgan

Date Submitted: March 30th, 2026

Professor: Daniel Shapiro

Faculty of Engineering  
Department of Computer Engineering  
2026

## Table of Contents

|                                       |              |
|---------------------------------------|--------------|
| <b>1. Overview.....</b>               | <b>2</b>     |
| <b>2. Literature Review.....</b>      | <b>2-3</b>   |
| I. Small object detection.....        | 2            |
| II. Motion Tracking.....              | 3            |
| <b>3. Methodology.....</b>            | <b>3-5</b>   |
| I. Experimental Philosophy.....       | 3            |
| II. Experiments.....                  | 3-5          |
| <b>4. System Design.....</b>          | <b>5-9</b>   |
| I. Dataset.....                       | 5-6          |
| II. Model Architecture.....           | 6-7          |
| III. System Architecture.....         | 8-9          |
| <b>5. Results &amp; Analysis.....</b> | <b>9-14</b>  |
| I Training Results & Analysis.....    | 9-13         |
| II. System Results & Analysis.....    | 13-14        |
| <b>6. Conclusion.....</b>             | <b>14</b>    |
| <b>7. Bibliography.....</b>           | <b>15-16</b> |

## **1. Overview**

The goal of this project was to implement a computer-vision based system capable of tracking a baseball through a video clip, and delivering an overlay, as well as metrics such as x-y break, velocity, and strike detection. The apparent issue of this model is that of small-object detection, an issue which has not been adequately resolved [7]; the primary cause of these issues comes from the low real-estate which the object holds within an image or frame [7].

Due to the constraints of small object detection, several different models and techniques were tried, as will be shown later in this paper. The eventual core of the detector was founded on a fine-tuned YOLOv12n model fine-tuned on a large dataset of augmented images.

The derivation of tracking and metrics from this model was a secondary focus of this project, and underwent a number of versions with varying degrees of success. Tracking was attempted using kalman filtering, quadratic interpolation, and the SORT algorithm. Break metrics and velocity were directly from the timing and pixel data, but were plagued by missed detections and varying camera angles.

## **2. Literature Review**

### **I. Small Object Detection**

The challenges of small object detection can be traced to three root causes: feature layers do not contain sufficient information, small objects contain limited contextual information, and a lack of positive examples for said small objects [7]. Detail of feature maps can be somewhat resolved by utilizing architectures that generate more complex feature maps than that of an CNN. Modern YOLO architectures contain several methods of improving feature maps, including multiscale feature fusion, FPN implementation, and attention mechanisms [8]. Contextual information can be addressed by increasing the size of the detection box, thus taking in more contextual information [7]. A lack of positive examples may be mitigated using an anchorless architecture such as YOLOv12 [8], multi-scale mechanisms, adapting matching, or generating overlapping anchors in the region proposal stage [7]. It can be concluded that the most promising established models for a small object detection task are faster RCNN, and later YOLO architectures.

## II. Motion Tracking

A variety of both classical and deep-learning based tracking methods have already been developed, including SORT, DeepSort, and StrongSort. The SORT algorithm combines a detection engine and several tracking techniques into a larger system. The system takes detections from an external model, uses Kalman filtering to predict the next value based on pixel velocity and position, and then uses the Hungarian algorithm for optimal matching based on the IoU of the detection and Kalman predictions [10]. DeepSORT utilizes the same Kalman and Hungarian method, but also implements a CNN to look at visual similarity between detections and interpolations [11]; the added overhead of the CNN renders this algorithm markedly slower, but more accurate. StrongSORT is similar to DeepSORT in that it also uses a neural network to improve matching, but adds some key improvements. Firstly, it utilizes a RE-ID network, improving the distinction between multiple tracked objects. StrongSORT also implements a more robust multi-stage matching strategy [12].

## 3. Methodology

### I. Experimental Philosophy

Through the experimentation phase of this project, an iterative experimentation paradigm was utilized to test the various contenders for final implementation. As each new potential solution arose, it was implemented and tested to check for potential viability. These tests primarily consisted of qualitative judgement of the given results, with some quantitative analysis performed on particularly promising contenders.

### II. Experiments

*II.a: CHT Detection.* While it does not exactly fit within the ML-AI scope of the project, this experiment is worth mentioning. Early tests were performed using the method of a circular hough transform, a classical CV method utilizing an edge-detector, combined with a voting system over all possible radii to determine whether or not an image feature is circular. This was found to be a quite computationally heavy process due to the large number of small circle checks required over a complex image, often taking several seconds per frame. In addition to this, this method was found to be quite sensitive to blur and noise, thus testing was stopped early.

II.b: SAM2. SAM2 is a segmentation model developed by Microsoft which develops full segmentation of a target image. While powerful, this model was found to be extremely computationally intense, and thus was initially to be used as an automatic annotation generator. Problems arose once again from the small scale and motion-blur of baseballs in video footage, leading to frequent missed or noisy segments. Detection of circular segments was attempted with the aforementioned CHT method, however the high degree of noise rendered this unreliable.

II.c: Pre-Trained YOLOvX. Several different ultralytics YOLO models were tested using the “sports-ball” class for the sake of baseball detection. Preliminary testing showed many dropped detections, however dropped detections were consistent and there were few false positives at a confidence threshold of 0.25. Earlier YOLO versions (YOLOv4) [9], had qualitatively worse overall detection than the later anchor-free versions (YOLOv12) [9]. Due to these promising results, additional experimentation in fine-tuning this model proceeded.

II.d: Fine-Tuned YOLOv12. Three fine-tuning runs were performed using the ultralytics library’s YOLOv12n pre-trained model. The first fine-tuning run was performed utilizing approximately 4000 training images [1] to get a baseline for performance. This model showed strong performance in the primary metrics of mAP50 and mAP50-95, with most late training runs (50 epochs) showing values of around 0.9 and 0.6 respectively. As this model was still seeing gains in several metrics, and had issues with generalization, an additional training run was performed. A much larger composite dataset was collected [1],[2], which was trimmed to a random 80-20 split of positive-negative examples, and new data was given the same augmentations as the previous run, including stretch to 640x640, contrast normalization, random crop (0-15%), random exposure adjustment ( $\pm 9\%$ ), random brightness shift ( $\pm 25\%$ ), and 5% coverage random noise. The run was performed with a total of 100 epochs to ensure convergence. This model received marked improvements on the primary metrics from the previous test, with mAP50 and mAP50-95 showing 0.959 and 0.658 respectively. Issues with generalization still appeared with non-tested and non-trained data, particularly with the early frames of videos with closer camera angles, necessitating an additional fine-tuning run. The third and final fine-tuning run took 2000 images from the previous dataset, as well as 3000 images altered from the previous set to add an additional 10-20% random crop. This fine-tuning run built off the results of the previous, using a slower learning rate and a reduced run of 30 epochs. This model showed a slight decrease in the primary metrics, with scores of 0.954 and 0.647 on the

mAP50 and mAP50-95 respectively, however showed an increase in generalization in qualitative testing. As the last two runs resulted in the final model, they will be discussed more in-depth in the results section.

100 epochs completed in 21.695 hours.

|       |        |           |       |       |       |                 |       |         |       |
|-------|--------|-----------|-------|-------|-------|-----------------|-------|---------|-------|
| Class | Images | Instances | Box(P | R     | mAP50 | mAP50-95): 100% | 16/16 | 0.4it/s | 37.6s |
| all   | 487    | 297       | 0.928 | 0.926 | 0.959 | 0.658           |       |         |       |

Figure 3.II.1: YOLOv12n Fine-Tuning Run 2 Summary

30 epochs completed in 4.108 hours.

|       |        |           |       |       |       |                 |       |         |       |
|-------|--------|-----------|-------|-------|-------|-----------------|-------|---------|-------|
| Class | Images | Instances | Box(P | R     | mAP50 | mAP50-95): 100% | 16/16 | 0.5it/s | 32.4s |
| all   | 487    | 297       | 0.905 | 0.934 | 0.954 | 0.647           |       |         |       |

Figure 3.II.2: YOLOv12n Fine-Tuning Run 3 Summary

*II.e: Additional Experiments.* A number of smaller, additional experiments were performed regarding preprocessing, and tracking. Firstly, an attempt was made to utilize the elimination of pixels which are not close to white from the frame to reduce background noise. This posed several issues with lighting variance, with no configuration of sensitivity settings being found such that the target object would not be eliminated from the frame, and a significant proportion of the frame was removed. This may be possible to remedy with well-implemented normalization techniques, however no suitable techniques were found. Tracking was attempted using simple quadratic regression of all detected points, however this was found to be ineffective. Pure Kalman filtering was also attempted, and while this worked well for short bursts of missed detections, large gaps in detections (> 3 frames) caused visible inaccuracy, and the tracker would trail off from the target-object after it had disappeared. SORT was eventually utilized, but due to time constraints the implementation was never perfected. Eventually a simple linear interpolation was used, which did not yield the most accurate results, but was by far the most stable solution.

## 4. System Design

### I. Dataset

The primary dataset constructed for the training of this project contains a combination of two separate datasets [1],[2], each of which contains a single class, “baseball”. These datasets

were combined into a singular dataset and normalized such that all annotations utilize the YOLO format, with the class “baseball” being a new class of index 0. All of these images underwent a preprocessing pipeline including stretch resizing to 640x640, and LAB contrast equalization. All images were then augmented with a random crop of 0-15%, random brightness adjustment of  $\pm 25\%$ , random exposure adjustment of  $\pm 9\%$ , and 5% salt and pepper noise injection. One of the source datasets included a random rotation augmentation, however this was not added to the other data sources. The resultant intermediate dataset contained a test/train/validate split of 1263/29116/784. In order to address the class imbalance of this dataset, negative examples were pruned so as to result in a 80-20 positive-negative split; the resultant final dataset had a test/train/valid split of 1017/17742/438.



Figure 4.I.1: Full Dataset Examples [1],[2]

An additional dataset was required to address some object scale issues with the original training run. The original dataset was sampled randomly for 5000 training images, for which 3000 had a random 10-20% crop applied. The test and validation splits remained the same however 60% of examples had the same 10-20% random crop applied. This resulted in a 1017/5000/438 test/train/validation split.

## II. Model Architecture

As discussed previously, the model architecture chosen for this project was YOLOv12 [8]. This neural network consists of three main structures, the backbone, the neck and the head. The backbone layer is responsible for initial feature extraction, and contains convolutional blocks, cross stage partial modules, shortcut connections, and attention layers.

The Convolutional blocks are responsible for the classical CNN-style feature extracting, performing the convolution + batch normalization + activation process to extract low to high level features. The cross-stage partial modules split the feature map into two pathways, one for

deep processing, another for shallow processing; these layers are then merged resulting in a more full feature map than a conventional CNN. The shortcut connections exist as a countermeasure to the vanishing gradient problem. An important addition to YOLOv12 is its attention mechanisms, allowing the network to generate more detailed features where an object is likely to appear. [8]

The neck of the neural network is responsible for feature fusion, and implements a combination of a feature pyramid network, and a path aggregation network. The FPN acts as the top-down pathway, upscaling high-level features for fusion with the lower level features. The PAN acts as the bottom-up pathway, reinforcing the localization signals and improving spatial data. [8]

The head of the YOLOv12 architecture uses an decoupled, anchor-free method rather than the anchor-based, coupled approach of early YOLO. This entails that rather than predicting based on predefined regions and offsets thereof, v12 is directly capable of predicting the width and height of the target-object. This unit predicts on three different stride levels (8,16,32), for different scales of objects. As this is a decoupled head, it has separate branches for class and bounding box predictions. [8]

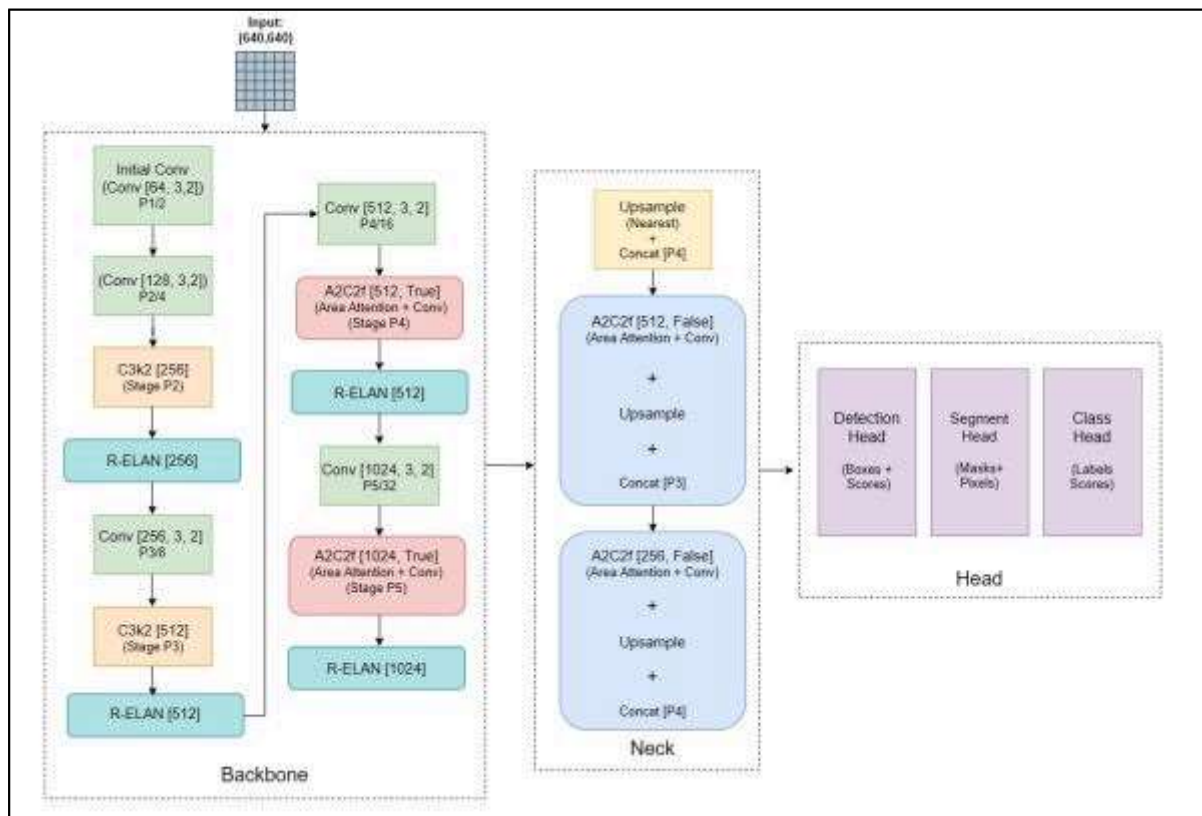


Figure 4.II.1: YOLOv12 Architecture [9]



### III. System Architecture

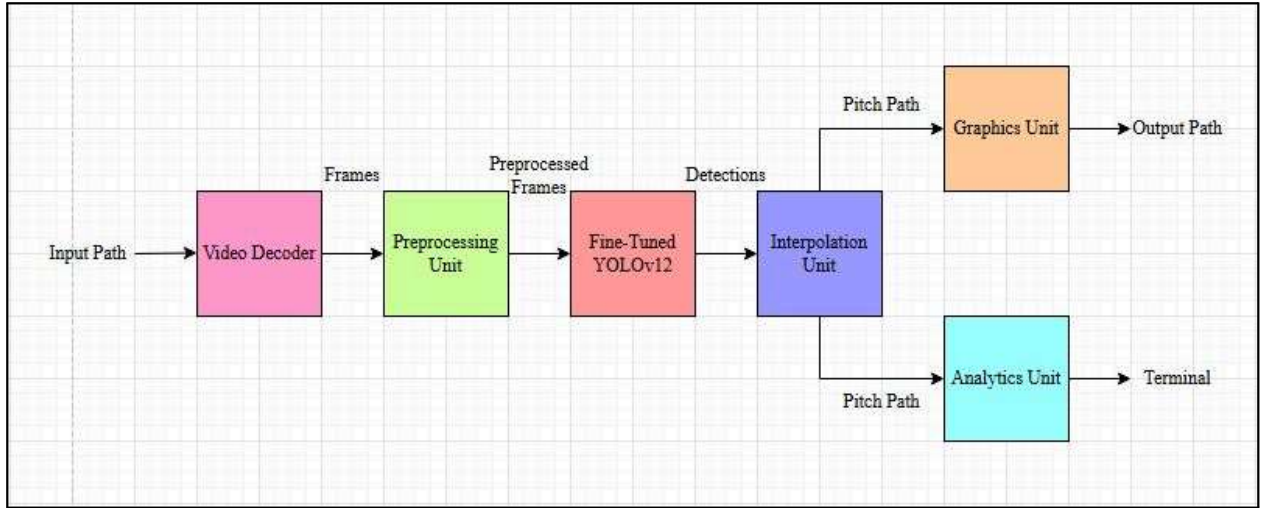


Figure 4.III.1: System Architecture

The system designed for this task was a rather simple, mostly linear processing pipeline. This pipeline consisted of five main units, the video decoder, the preprocessing unit, the YOLOv12 detection model, the interpolation unit, and parallel units for video graphics and pitch analytics.

The first of these units, the video decoder unit, was rather simple. This unit simply used openCV's [3] video capture feature to generate a stream of frames which can be accessed in order by the preprocessing unit. The preprocessing unit then takes said stream and applies a series of visual transforms to replicate parts of the preprocessing as used in the dataset. This unit applies an `exif_transpose`, stretch-resizes the frame to 640x640, and performs LAB normalization before returning the image. This unit is called by the detections unit to pull processed frames one by one into the YOLOv12 detection model, and place both the detections and the frames into arrays to be passed to the interpolation unit. The interpolation unit first iterates over all frames, keeping only the best detection from each frame via a combination of the box confidence score, and a distance threshold. From these best detection an interaction is done to find any gaps in detection. For each of these detection gaps, the pixel difference between these detections is converted to a stride based on the size of the frame gap, and a new point is inserted at the point of  $(\text{stride} \times \text{frame number})$ . Finally, this continuous path is forked to two units, the analytics module and the graphics module. The graphics module simply takes each frame of the video, draws a circle on each point as they appear, and saves a new video containing a tracer over

the baseball. The analytics unit takes information about the pitch path, and derives statistics such as ball speed.

## 5. Results & Analysis

### I. Training Results & Analysis

Using the dataset discussed in section 4.I, a YOLOv12n model was fine tuned utilizing the Ultralytics library [5]. This training was conducted in two runs, a large base run (approx 18,000 training images), and a smaller run to fix any generalization issues with the previous run (approx 5000 images). No custom testing or validation scripts were required, as Ultralytics performs this function automatically.

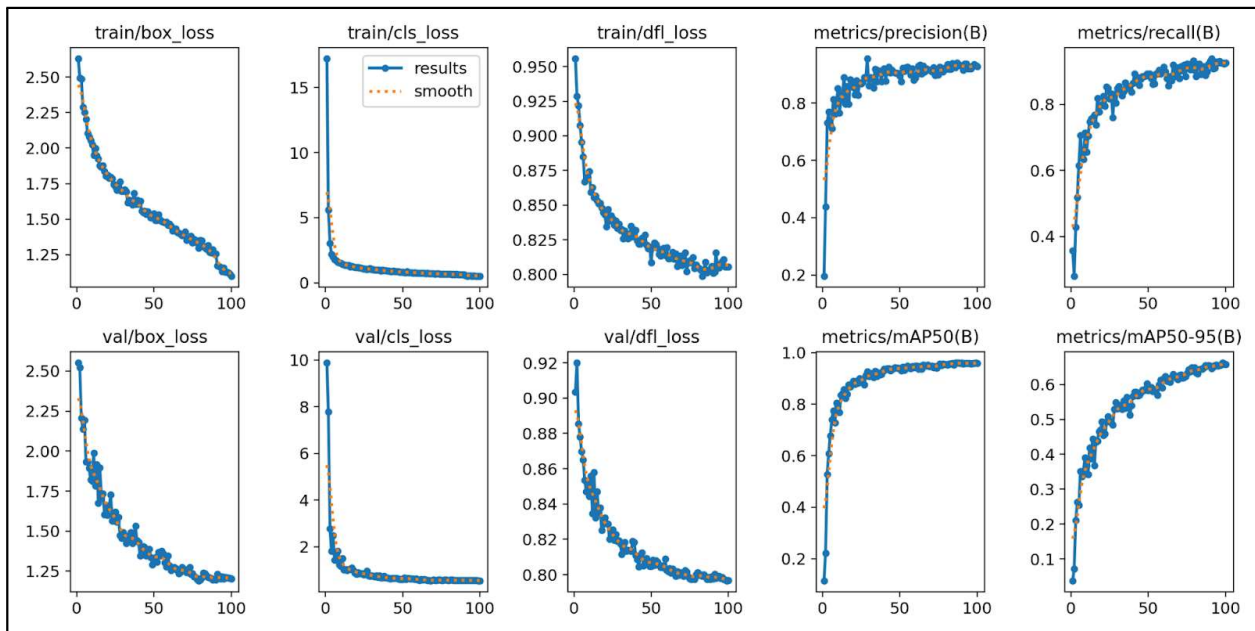


Figure 5.I.1: Training Run 1 Results

This first large training run was able to lay out the groundwork of the model quite effectively, with all metrics falling within at minimum acceptable results. The main metrics which were focused on were the mAP50, and the mAP50-95, denoting the mean average precision of the model under two different IOU constraints. The best run for these metrics came in at mAP50 = 0.959, and mAP50-95 = 0.658. The 0.95+ value of mAP50 showed that the model was getting at minimum an acceptable detection in 95%+ of the tested examples, with the mAP50-95 indicating more than half of examples showed a very good detection. The loss curves (box\_loss, cls\_loss, dfl\_loss) were used as an indicator regarding the potential improvement in

different aspects additional epochs could provide. `cls_loss`, as predicted, leveled out very quickly (around epoch 25), as there was only one class excluding background. The `dfl_loss` and `box_loss` were shown to be the limiting factor, however they did plateau towards the end of training (between epochs 50 and 100), indicating minimal gain could be made through additional training time.

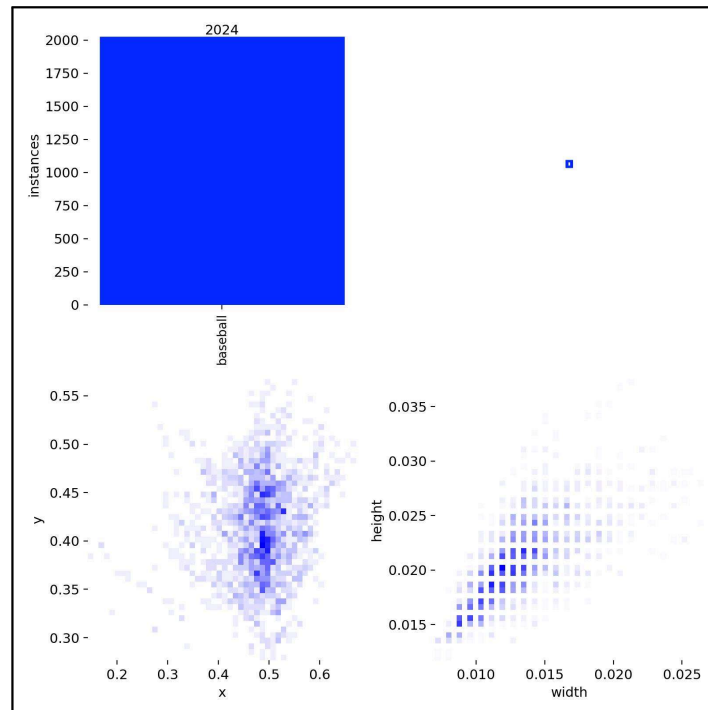


Figure 5.I.2: Run 1 Detection Heatmap

The detection heatmap as shown in figure 5.I.2 gives insight into where spatially a ball was detected in testing. It is shown that balls were detected mostly centred, with a slight skew to the right of the frame. This holds with the expected detection, as the majority of pitchers are right handed, and thus would throw around the centre-right of the frame. The chart to the right of the heatmap indicated the width-height concentration of bounding boxes generated by the model. This distribution indicated an issue that was confirmed by real-world testing, as many detections were dropped early in a pitch as the ball was too close to the camera.

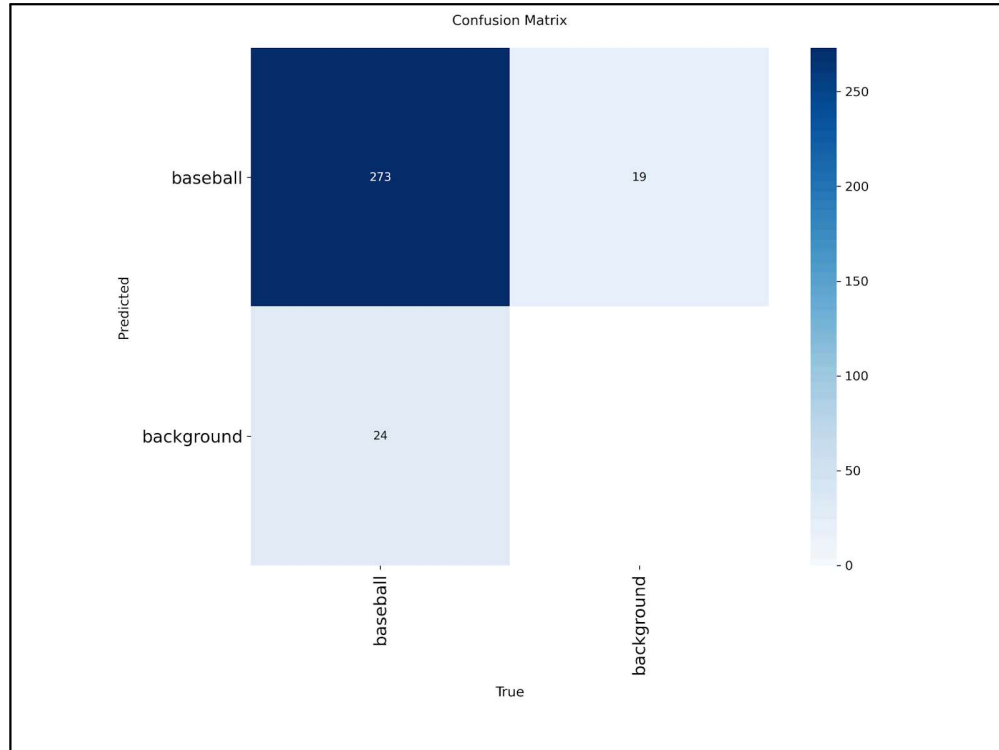


Figure 5.I.3: Run 1 Confusion Matrix

The confusion matrix visually indicated the number of examples with correct detections, incorrect detections, and missed detections. As shown, a total of 273 of tested examples were correct to some degree, and a total of 43 were missed. The number of misses was fairly evenly distributed between the two categories, showing a good precision-recall tradeoff.

Due to the box scaling issue of the first run, a second run was performed to further fine-tune the resultant model. This was performed with the smaller, 5000 training image dataset, and some adjustments to the training script. A lower learning rate and a freeze of some layers ensured that results from the previous run would not be destroyed

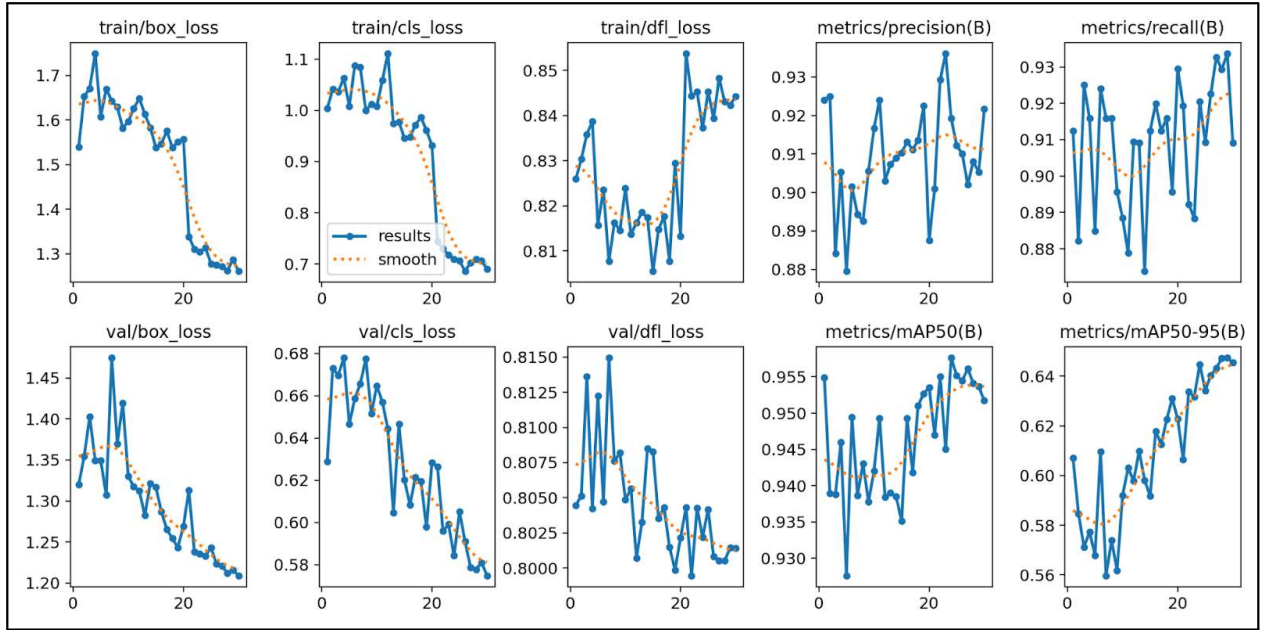


Figure 5.I.4: Training Run 2 Results

Due to the smaller scale of this training run, the metric curves show a noisier pattern than the previous. The mAP50 and mAP50-95 show slightly depreciated results, with values of 0.954 and 0.646. Despite the drop, these values were interpreted as a very slight drop provided generalization did in fact increase. The noise of the loss curves rendered it somewhat difficult to determine if additional training was warranted, however as the mAP results began to level off, it is unlikely.

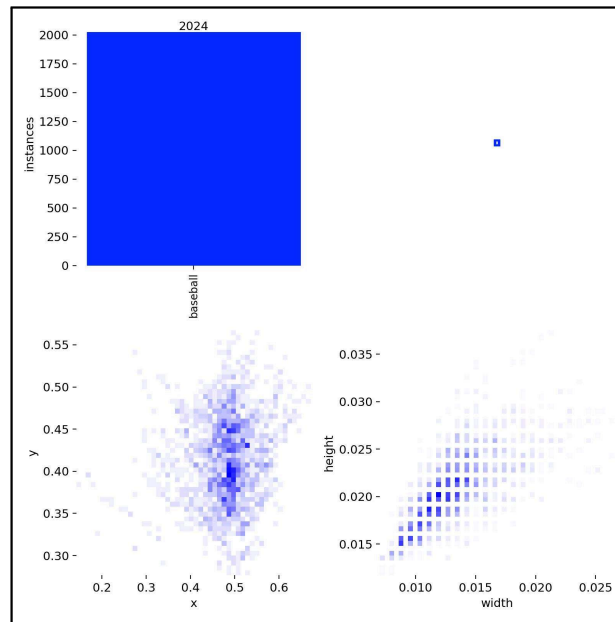


Figure 5.I.5: Run 2 Detection Heatmap

The detection heatmap results of the second run are largely the same, with some notable differences. Firstly, the spatial detection heatmap is nearly identical, which was the desired result as pitches tend to stay largely within the same x-y tunnel throughout their duration. The box size heatmap however showed larger concentration further down the x-y diagonal, showing that larger boxes were being detected while retaining their square shape. This suggested improved generalization, which was corroborated by informal testing.

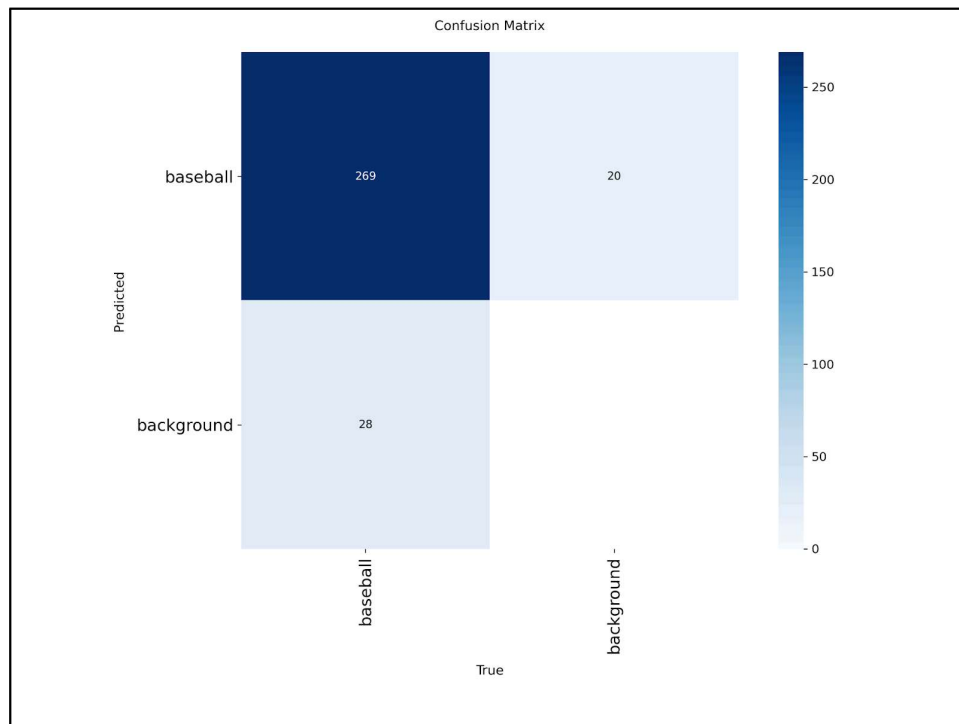


Figure 5.I.6: Run 2 Confusion Matrix

The confusion matrix stayed largely the same, with the distribution of false detections skewing slightly towards baseball detections being missed. The shift was a small one, and can likely be attributed to noise.

## II. System Results & Analysis

Minimal concrete testing was performed on the system as a whole due to unforeseen timing constraints, however observations were made on individual examples which gleaned insight as to future directions for improvement. Firstly, issues in normalization caused significant issues in system consistency. The system was observed to function properly for almost all pitch video examples however under different circumstances. The confidence threshold in prediction shows major variance in ideal performance, with examples showing their best results anywhere

between a 0.01 and 0.001 threshold value. Issues in early frame detection persisted throughout versions 1 and 2 of the model, though to a significantly lesser degree. Erroneous detections near to the latest detection would occasionally cause catastrophic failure, as if they were higher confidence than the more true prediction, the interpolation would lead the interpolation algorithm significantly astray. Issues with the analytics module also caused it to remain mostly non-functional. Pitch speed was determined by the number of frames of the trajectory and an assumed pitch distance of 55.5 feet, however some pitchers inherently release farther towards the plate than others, skewing velocity calculation. The early detections issue also caused issues with velocity calculation, as they cut off the first few frames of the pitch's trajectory, thus skewing the pitch faster than it should have been. Due to these factors, several examples displayed velocities well exceeding 100mph.

## **6. Conclusion**

The results of this project were somewhat of a mixed bag, with the machine learning core being in a very workable state, and the surrounding infrastructure lacking consistency and some features. The fine-tuned YOLOv12 tracking model effectively detects the baseball throughout the majority of a pitches duration, giving in theory adequate information to produce a continuous pitch trajectory. The model would still benefit from an increased dataset domain, particularly more amateur footage, and more varied camera angles. In addition to this, increased functionality could be added by including classes for home-plate and batters, allowing a strike zone to be generated and ball/strike calls to be inferred. The interpolation system, while it works in part, would benefit from additional recovering options and trajectory smoothing. Additional metrics also be implemented, such as the aforementioned ball/strike inferences, as well as x-y pitch break. Overall, the foundations of the project are very strong, however it lacks some key refinements which could be afforded by additional development time.

## 7. Bibliography

- [1] J. Holm, "Baseball-Identification," Hugging Face Datasets, 2024. [Online]. Available: <https://huggingface.co/datasets/Jensen-holm/Baseball-Identification>
- [2] R. Sanchez, "baseball-gemqh (Version 1)," Roboflow Universe, 2024. [Online]. Available: <https://universe.roboflow.com/ruben-sanchez-mmnl/baseball-gemqh/dataset/1>
- [3] OpenCV, "OpenCV: Open Source Computer Vision Library," version 4.11.0, Software, 2023. [Online]. Available: <https://opencv.org/>
- [4] NumPy, "NumPy," version 2.4.1, Software, 2023. [Online]. Available: <https://numpy.org/>
- [5] G. Jocher, A. Chaurasia, and J. Qiu, "YOLO by Ultralytics," version 12.0.0, Software, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [6] A. Paszke, S. Gross, F. Massa, et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," version 2.8.0, Software, 2024. [Online]. Available: <https://pytorch.org/>
- [7] Y. Liu, P. Sun, N. Wergeles, and Y. Shang, "A survey and performance evaluation of deep learning methods for small object detection," *\*Expert Systems with Applications\**, vol. 172, Art. no. 114602, pp. 1–14, Jun. 2021.
- [8] R. Khanam and M. Hussain, "A review of YOLOv12: Attention-based enhancements vs. previous versions," arXiv preprint arXiv:2504.11995, Apr. 2025.
- [9] N. Jegham, C. Y. Koh, M. Abdelatti, and A. Hendawi, "YOLO evolution: A comprehensive benchmark and architectural review of YOLOv12, YOLO11, and their previous versions," arXiv preprint arXiv:2411.00201, 2024.
- [10] I. C. B. Fernandez, P. C. Magpantay, M. D. Rosales, and J. R. E. Hizon, "Review of Kalman filters in multiple object tracking algorithms," in *\*Proc. 2024 IEEE Int. Conf. on Omni-layer Intelligent Systems (COINS)\**, 2024, pp. 1–4, doi: 10.1109/COINS61597.2024.10622143.
- [11] F. Yang, X. Zhang, and B. Liu, "Video object tracking based on YOLOv7 and DeepSORT," arXiv preprint arXiv:2207.12202, 2022. [Online]. Available: <https://arxiv.org/abs/2207.12202>.



- [12] M. M. Khalil, S. N. Saleh, N. S. Tawfik, and M. N. Elagamy, "EF-StrongSORT: An Enhanced Feature StrongSORT Model for Multi-Object Tracking," *IEEE Access*, vol. 13, pp. 53608–53620, Mar. 2025, doi: 10.1109/ACCESS.2025.3554706.